Algorithmes et Pensée Computationnelle

Programmation orientée objet - Exercices basiques

Le but de cette séance est de découvrir et de mettre en pratique un paradigme de programmation largement utilisé : la programmation orientée objet (POO). Ce paradigme repose sur la création d'objets qui regroupent des données et des comportements, et qui peuvent interagir entre eux. Au fil des exercices, vous apprendrez à manipuler des objets et à maîtriser les notions fondamentales de classe, d'attribut, de méthode et d'encapsulation. À l'issue de cette séance, vous serez capables de concevoir des programmes mieux structurés et plus faciles à maintenir.

Les exercices de cette séance seront réalisés en Java et en Python.

Les durées indiquées (🕓) sont données à titre indicatif.

1 Itération en Java

Question 1: (🕓 *5 minutes*)

> **Conseil**
>
> La méthode `String.valueOf` permet de convertir un nombre en chaîne de caractères.

Qu'affiche le programme Java suivant (par souci de concision, nous omettons la déclaration de la classe et de la méthode `main`) :

```
1 String result = "";
2 String x = "5";
3 int j = 0;
4 for (j=Integer.parseInt(x); j !=0; j = j/2){
5     result = String.valueOf(j%2) + result;
6 }
7 System.out.println(result + j);
```

Choisissez parmi les éléments suivants :

(A) `5210`

(B) `1110`

(C) `1010`

(D) `1011`

(E) `java: bad operand types for binary operator '+'..`

2 Récursion en Java et Python

Question 2: (🕓 *10 minutes*) [Java et Python] **isPalindrome**

> **Conseil**
>
> En Java, pour obtenir le caractère à l'indice i d'une chaîne de caractères, `myString`, utilisez `myString.charAt(i)`. En Python, utilisez `myString[i]`.
>
> Une sous-chaîne est obtenue en Java avec `myString.substring(i, j)` où i est l'indice du premier caractère et $j-1$ est l'indice du dernier caractère. En Python, utilisez `myString[i:j]`.

Un palindrome est une chaîne de caractères qui se lit de la même manière à l'envers. Plus précisément, soit f une fonction à valeurs booléennes qui renvoie vrai si `a` (de longueur n caractères) est un palindrome :

$$f(a) = f(a[1 : n-1]) \wedge (a[0] == a[n-1]) \tag{1}$$

Ici, $a[1 : n-1]$ est la sous-chaîne de a qui comprend tous les caractères à l'exception du premier caractère (le premier caractère étant à l'indice 0) et du dernier caractère (le dernier caractère étant à l'indice $n-1$).

Conseil

Revoir la question 16 du TP3 pour voir les règles concernant la notation des bornes d'une sous-chaîne de caractères.

On dit donc que a est un palindrome si le premier caractère et le dernier caractère sont égaux **et** la sous-chaîne entre les deux est un palindrome.

Le cas de base (une chaîne a d'un seul caractère) est :

$$f(a) = \text{vrai si } n = 1$$

et le cas de base d'une chaîne vide : $f(\texttt{""}) = \text{vrai}$.

Écrivez la méthode Java **`isPalindrome(String arg)`** et la fonction Python **`isPalindrome(arg)`** qui renvoient la valeur booléenne vraie si `arg` est un palindrome : `true` en Java et `True` en Python.

3 Création de votre première classe en Java

Le but de cette première partie est de créer votre propre classe en Java. Cette classe, nommée `Dog`, représentera un chien. Elle aura plusieurs attributs et méthodes que vous implémenterez au fur et à mesure.

Question 3: (*10 minutes*) Création de classe et encapsulation
Commencez par créer une nouvelle classe `Dog` dans votre projet. Ensuite, créez les attributs suivants :

1. Un attribut `public` String nommé `name`
2. Un attribut `private` List nommé `tricks`
3. Un attribut `private` String nommé `race`
4. Un attribut `private` int nommé `age`
5. Un attribut `private` int nommé `mood` initialisé à 5 (correspondant à l'humeur du chien)
6. Un attribut de classe (`static`) `private` int nommé `nbChiens`

Déclarez le constructeur public de la classe `Dog`. En Java, un constructeur porte le même nom que la classe et ne possède aucun type de retour. Il sert à initialiser les nouvelles instances. Le constructeur prendra les paramètres suivants :

1. Une chaîne de caractères `name`,
2. Une liste `tricks`,
3. Une chaîne de caractères `race`,
4. Un entier `age`.

Pour finir, le constructeur doit incrémenter l'attribut de classe `nbChiens`, qui garde en mémoire le nombre d'instances créées.

Conseil

Pour cet exercice, n'oubliez pas de préciser si vos attributs sont `public` ou `private`.
Le mot `static` correspond à un élément de classe (attribut ou méthode), cet élément pourra ensuite être appelé via la classe directement.
Pour attribuer des valeurs à vos attributs d'instance, utilisez le mot-clé `this.attribut`.
Pour accéder aux attributs de classe, utilisez `nomClasse.nomAttribut`.

Question 4: (*10 minutes*) Getters et setters
Il faut maintenant créer des méthodes de type `getter` et `setter` afin d'interagir avec les attributs

`private` des instances de la classe. Les `getters` renverront les attributs souhaités tandis que les `setters` les modifieront.
Les `setters` sont souvent utilisés pour modifier la valeur d'attributs privés et ne renvoient rien.

Conseil

Exemple de `getters` et de `setters` :

```
1 public String getName() { // Exemple de getter
2     return name;
3 }
4
5 public void setName(String name) { // Exemple de setter
6     this.name = name;
7 }
```

Vous pouvez directement accéder à des attributs publics en utilisant `nomInstance.attribut` à l'intérieur ou à l'extérieur de la classe.

Créez les méthodes suivantes :

- `public List getTricks()`
- `public String getRace()`
- `public int getAge()`
- `public int getMood()`
- `public void setTricks(List tricks)`
- `public void setRace(String race)`
- `public void setAge(int age)`
- `public void setMood(int mood)`

Créez également la méthode de classe `public static int getNbChiens()`, qui retourne le nombre de `Dog` instanciés.

Conseil

Intellij vous permet de générer automatiquement certaines méthodes telles que les getters et setters. Vous pouvez consulter le lien suivant pour plus d'informations : `https://www.jetbrains.com/help/idea/generating-code.html#generate-delegation-methods`.
Toutefois, pour cet exercice, nous vous encourageons à le faire manuellement.

Question 5: (*5 minutes*) Manipulation d'attributs - Listes
Créez une méthode publique nommée `addTrick(String trick)` qui prend en entrée une chaîne de caractères et l'ajoute à la liste `tricks`.

Conseil

La liste `tricks` est une liste comme les autres. Si vous voulez la modifier, vous aurez besoin de passer par une `LinkedList` temporaire.

Question 6: (*5 minutes*) Manipulation d'attributs - `setter`
Créez deux méthodes permettant de modifier l'attribut `mood` de l'objet `Dog`. La méthode `leash()` décrémentera `mood` de 1 et `eat()` l'incrémentera de 3.

Question 7: (*5 minutes*) Manipulation d'attributs d'une autre instance
Créez une méthode nommée `getOldest(Dog other)` qui prend comme argument un élément de type `Dog`, puis retourne le nom et l'âge du chien le plus âgé au format suivant : "`nomChien` est le chien le plus âgé : il a `ageChien` ans". Vous pouvez supposer que les deux chiens ont des âges différents.

> **Conseil**
>
> L'élément `other` est un objet de type `Dog` : vous pouvez donc appeler les méthodes définies précédemment pour consulter ses attributs.

Question 8: (*5 minutes*) Redéfinition de méthodes
Redéfinissez la méthode `public String toString()` afin qu'elle retourne une chaîne de caractères contenant toutes les informations d'une instance de `Dog`. Ainsi, dans votre `main`, en affichant sur la console une instance de `Dog`, vous obtiendrez un texte au format suivant : "`nomChien` a `ageChien` ans, est un `raceChien` et a une humeur de `moodChien`. Il sait faire les tours suivants : `tricksChien`".

> **Conseil**
>
> La méthode `toString()` permet d'obtenir une représentation textuelle d'un objet. Elle est définie par défaut pour tout objet. Pour la redéfinir, on utilise le mot-clé `@Override` avant la méthode.

Pour contrôler que vos méthodes et attributs ont été implémentés correctement, exécutez le programme suivant :

```
1  import java.util.List;
2
3  public class Question8 {
4      public static void main(String[] args) {
5          Dog lola = new Dog("Lola",List.of("rollover"),"Bouvier",10);
6          Dog tobi = new Dog("Tobi",List.of("rollover","do a
       barrel"),"Doggo",17);
7          System.out.println(lola.getAge());
8          System.out.println(lola.getMood());
9          System.out.println(lola.getRace());
10         System.out.println(lola.name);
11         System.out.println(lola.getTricks());
12         lola.setAge(13);
13         lola.setMood(8);
14         lola.setRace("Bouvier");
15         lola.name = "lola";
16         lola.setTricks(List.of("rollover","do a barrel"));
17         lola.eat();
18         lola.leash();
19         lola.addTrick("sit");
20         System.out.println(Dog.getNbChiens());
21         System.out.println(lola.getOldest(tobi));
22         System.out.println(lola);
23     }
24 }
```

Vous devriez obtenir ce résultat :

```
1  10
2  5
3  Bouvier
4  Lola
5  [rollover]
```

```
6 2
7 Tobi est le chien le plus âgé : il a 17 ans
8 lola a 13 ans, est un Bouvier et a une humeur de 10. Il sait faire les
      tours suivants : [rollover, do a barrel, sit]
```

4 Notions de POO en Python

Dans cette section, nous créerons pas à pas une classe `Point` contenant des attributs et des méthodes utiles. Dans votre IDE, créez un nouveau projet Python (Fichier > Nouveau > Projet). Dans un dossier de votre choix, créez un fichier `question9.py`.

Question 9: (*15 minutes*) Classe `Point`

- Créez une classe `Point` et définissez un constructeur prenant deux paramètres, `x` et `y`.

> **Conseil**
>
> Pour rappel, un constructeur est une fonction `__init()__` que vous redéfinirez dans votre classe.

- Définissez deux attributs privés pour votre classe `Point`. Ces attributs seront les coordonnées x et y de vos points. Par défaut, assignez-leur les valeurs données dans le constructeur.

> **Conseil**
>
> À l'intérieur d'une classe, utilisez le mot-clé `self` pour accéder aux méthodes et attributs de l'instance que vous manipulez. En Python, pour spécifier qu'un attribut est privé, ajoutez deux traits de soulignement au nom de l'attribut (exemple : `__score=0`).

- Définissez les getters et les setters.

> **Conseil**
>
> En Python, le mot-clé `self` est l'équivalent de `this` utilisé en Java.

- Définissez une méthode `distance` qui prend en entrée l'instance du `Point` (`self`) et un autre `Point` `p2`. Cette méthode `distance` retournera la distance euclidienne entre le point `self` et `p2`.

> **Conseil**
>
> Pour rappel, la distance euclidienne entre deux points est définie par la formule $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Utilisez la fonction `sqrt()` de la bibliothèque `math` pour calculer la racine carrée. Pensez à importer la bibliothèque `math`.

- Définissez une méthode `milieu` qui prendra en entrée `self` et `p2` et qui retournera un objet `Point` situé entre `self` et `p2`.

> **Conseil**
>
> Pour trouver les coordonnées d'un point M(x_M, y_M) situé au milieu du segment défini par des points A(x_A, y_A) et B(x_B, y_B), utilisez les formules suivantes : $x_M = \frac{x_1+x_2}{2}$ et $y_M = \frac{y_1+y_2}{2}$

- Redéfinissez la méthode `__str__()` dans la classe `Point` afin qu'elle retourne ses coordonnées. Par exemple, pour un point de coordonnées $(3, 2)$, la chaîne retournée sera : *Les coordonnées du Point sont : x = 3 et y = 2*

5 Notions d'héritage - Java

Le but de cette partie est de mettre en pratique les notions liées à l'héritage. Nous allons créer une classe `Livre` qui représentera notre classe-mère. Nous allons également créer deux classes filles, `LivreAudio` et `LivreIllustre`. Les classes filles hériteront des attributs et méthodes de la classe-mère.

Question 10: (*20 minutes*) `Création des différentes classes`
Créez la classe-mère `Livre` avec les caractéristiques suivantes :

- un attribut `private String` nommé `titre`,
- un attribut `private String` nommé `auteur`,
- un attribut `private int` nommé `annee`,
- un attribut `private int` nommé `note` (initialisé à `-1`),
- le constructeur `public Livre(String titre, String auteur, int annee)`,
- une méthode `public void setNote(int note)` qui permet de définir l'attribut `note`,
- une méthode `public int getNote()` qui permet de retourner l'attribut `note`,
- une méthode `public String toString()` qui retournera le titre, l'auteur, l'année et la note d'un ouvrage (redéfinir cette méthode permettra d'afficher un objet `Livre` en utilisant `System.out.println()`).

Attention, si la `note` n'a pas été modifiée et qu'elle vaut toujours `-1`, affichez "Note : pas encore attribuée" au lieu de "Note : `note`" via la méthode `toString()`.
Créez les classes filles avec les caractéristiques suivantes :
`class LivreAudio extends Livre`

- un attribut `private String` nommé `narrateur`,
- le constructeur `public LivreAudio(String titre, String auteur, int annee, String narrateur)`, qui initialise les attributs hérités et l'attribut `narrateur`.

`class LivreIllustre extends Livre`

- un attribut `private String` nommé `illustrateur`,
- le constructeur `public LivreIllustre(String titre, String auteur, int annee, String illustrateur)`, qui initialise les attributs hérités et l'attribut `illustrateur`.

Voici le squelette du programme à compléter :

```
1 class Livre {
2
3 }
4 class LivreAudio extends Livre {
5
6 }
7 class LivreIllustre extends Livre {
8
9 }
```

Conseil

- En Java, lors de la déclaration d'une classe, le mot-clé `extends` permet d'indiquer qu'il s'agit d'une classe fille de la classe indiquée. Pour initialiser la partie héritée d'un objet, appelez le constructeur de la classe-mère avec `super(attributMere1, attributMere2, attributMere3, etc.);`.
- L'appel au constructeur `super(...)` doit toujours être la première instruction du constructeur d'une classe fille.
- Vous pouvez vous servir de `'\n'` dans une chaîne de caractères pour effectuer un retour à la ligne lors de l'affichage d'une chaîne de caractères.

Question 11: (*10 minutes*) **Méthode et héritage**
Maintenant que vous avez créé la classe-mère et ses classes filles, procédez comme suit :

1. Dans la classe `LivreAudio`, redéfinissez la méthode `toString()` afin qu'elle retourne les informations définies dans la classe `Livre`, suivies de la valeur de l'attribut `narrateur`.
2. Faites de même dans la classe `LivreIllustre` afin d'ajouter la valeur de l'attribut `illustrateur`.
3. Dans la méthode `main`, déclarez une variable de type `Livre` et initialisez-la avec une instance de `LivreAudio`. Utilisez les valeurs suivantes : titre `Hamlet`, auteur `Shakespeare`, année `1609` et narrateur `William`.
4. Attribuez la note `5` à ce livre à l'aide de la méthode setter définie précédemment.
5. Déclarez une deuxième variable de type `Livre` et initialisez-la avec une instance de `LivreIllustre`. Utilisez les valeurs suivantes : titre `Les Misérables`, auteur `Victor Hugo`, année `1862` et illustrateur `Gustave Brion`. Ne lui attribuez aucune note afin de vérifier le comportement prévu pour la valeur `-1`.
6. Affichez les informations des deux livres.

Conseil

Réfléchissez à la différence entre le type d'une variable et le type de l'objet qu'elle référence. Pour réutiliser le comportement de la classe parente dans une méthode redéfinie, pensez au mot-clé `super`.

Lorsque toutes les étapes auront été effectuées, exécutez le programme suivant :

```
1  public class Question11 {
2
3      public static void main(String[] args) {
4          Livre livreAudio = new LivreAudio("Hamlet", "Shakespeare",
       1609, "William");
5          livreAudio.setNote(5);
6          Livre livreIllustre = new LivreIllustre("Les Misérables",
       "Victor Hugo", 1862, "Gustave Brion");
7
8          System.out.println(livreAudio);
9          System.out.println(livreIllustre);
10     }
11 }
```

Vous devriez obtenir :

```
1  A propos du livre
2  ------------------
3  Titre : Hamlet
4  Auteur : Shakespeare
5  Année : 1609
6  Note : 5
7  Narrateur: William
8  A propos du livre
9  ------------------
10 Titre : Les Misérables
11 Auteur : Victor Hugo
12 Année : 1862
13 Note : pas encore attribuée
14 Illustrateur: Gustave Brion
```